

AUTOM-03 LAB: Monaco

Configuration as Code

Series: AUTOM — Dynatrace Automation | **Notebook:** 3 of 8 (LAB) | **Created:** April 2026 | **Last Updated:** 04/25/2026

Overview

Hands-on lab for installing Monaco CLI, downloading existing Dynatrace configuration, modifying it, validating, and deploying to a target environment. Includes CI/CD pipeline integration with GitHub Actions. Complete the lecture notebook **AUTOM-03: Monaco Configuration-as-Code** first, then work through these exercises step by step.

Table of Contents

1. [Install Monaco CLI](#)
 2. [Create Access Token](#)
 3. [Create Manifest File](#)
 4. [Download Existing Configuration](#)
 5. [Explore Downloaded Configuration](#)
 6. [Create a New Configuration](#)
 7. [Validate Configuration](#)
 8. [Deploy Configuration](#)
 9. [Delete Configuration](#)
 10. [Multi-Environment Promotion](#)
 11. [GitHub Actions CI/CD Pipeline](#)
 12. [Summary & Checklist](#)
-

Prerequisites

Requirement	Details
Completed	AUTOM-03: Monaco Configuration-as-Code (lecture notebook)
Dynatrace Environment	SaaS tenant (Grail enabled)
Permissions	Account with ability to create API tokens
Local Tools	Terminal with <code>curl</code> , <code>chmod</code> , or Homebrew (macOS)
Git	Git CLI installed (for CI/CD section)
GitHub Account	Required for Section 11 (CI/CD pipeline)

1. Install Monaco CLI

Monaco is a standalone binary with no runtime dependencies. Choose the installation method for your platform.

macOS (Homebrew)

```
# Install Monaco via Homebrew (macOS)
brew install dynatrace-oss/monaco/monaco
```

Linux / macOS (curl)

```
# Download the latest Monaco binary for Linux (amd64)
curl -L https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/latest/download/monaco-linux-amd64 -o monaco

# Make it executable and move to PATH
chmod +x monaco
sudo mv monaco /usr/local/bin/

# For macOS (Apple Silicon), use:
# curl -L https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/latest/download/monaco-darwin-arm64 -o monaco
```

Windows (PowerShell)

```
# Download Monaco for Windows
Invoke-WebRequest -Uri "https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/latest/download/monaco-windows-amd64.exe" -OutFile "monaco.exe"

# Move to a directory in your PATH
Move-Item monaco.exe C:\Tools\monaco.exe
```

Verify Installation

```
# Verify Monaco is installed and check version
monaco version
```

Note: Monaco releases are available at github.com/Dynatrace/dynatrace-configuration-as-code/releases. Check for the latest version before installing.

2. Create Access Token

Monaco requires an API token with specific scopes to read and write configurations.

Required Token Scopes

Scope	Purpose
<code>Read settings (settings.read)</code>	Download Settings 2.0 objects
<code>Write settings (settings.write)</code>	Deploy Settings 2.0 objects
<code>Read SLO (slo.read)</code>	Download SLO definitions
<code>Write SLO (slo.write)</code>	Deploy SLO definitions
<code>Read configuration (ReadConfig)</code>	Download classic API configs
<code>Write configuration (WriteConfig)</code>	Deploy classic API configs
<code>Access problem and event feed, metrics, and topology (DataExport)</code>	Entity and metric access
<code>Read entities (entities.read)</code>	Entity queries during download
<code>Create and read synthetic monitors, locations, and nodes (ExternalSyntheticIntegration)</code>	Synthetic config management

Tip: In the Dynatrace UI, navigate to **Access Tokens** and use the predefined template "**Configuration as Code**" to create a token with all required scopes automatically.

Set Environment Variables

```
# Set your Dynatrace environment URL and API token
# Replace with your actual values
export DT_ENV_URL="https://&lt;your-environment-id&gt;.live.dynatrace.com"
export
DT_API_TOKEN="dt0c01.XXXXXXX.YYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYY
YYYYYYYYYYYYYYYYYYY"

# Verify the variables are set
echo "Environment: $DT_ENV_URL"
echo "Token set: $([ -n \"$DT_API_TOKEN\" ] && echo 'yes' || echo 'no')"
```

Warning: Never commit API tokens to version control. Use environment variables or a secrets manager.

3. Create Manifest File

The `manifest.yaml` is the entry point for every Monaco project. It defines which projects to manage and which environments to target.

Project Directory Setup

```
# Create the project directory structure
mkdir -p dynatrace-config/my-project
cd dynatrace-config
```

Manifest Format

Create `dynatrace-config/manifest.yaml` with this content:

```

manifestVersion: "1.0"

projects:
  - name: my-project
    path: my-project

environmentGroups:
  - name: default
    environments:
      - name: dev
        url:
          type: environment
          value: DT_ENV_URL
        auth:
          token:
            name: DT_API_TOKEN

```

Field Reference

Field	Description
<code>manifestVersion</code>	Always <code>"1.0"</code> for current Monaco
<code>projects[].name</code>	Logical name for the project
<code>projects[].path</code>	Relative path to the project directory
<code>environmentGroups[].name</code>	Group name (used for overrides)
<code>environments[].name</code>	Environment identifier (used with <code>--environment</code> flag)
<code>environments[].url.value</code>	Environment variable name containing the tenant URL
<code>environments[].auth.token.name</code>	Environment variable name containing the API token

Important: The `url.value` and `auth.token.name` fields reference **environment variable names**, not the actual URL or token values. Monaco reads the values from your shell environment at runtime.

4. Download Existing Configuration

Downloading is the fastest way to get started. Monaco exports your current tenant configuration into versioned YAML and JSON files.

Download All Configuration

```
# Download all configuration from the dev environment
cd dynatrace-config
monaco download manifest.yaml --environment dev --output-folder downloaded
```

Download Specific Configuration Types

```
# Download only alerting profiles and management zones
cd dynatrace-config
monaco download manifest.yaml --environment dev --output-folder downloaded \
  --api builtin:alerting.profile,builtin:management-zones
```

What Gets Downloaded

Monaco organizes downloaded configuration by type:

Config Type	Description	Example Schema IDs
settings	Settings 2.0 objects	builtin:alerting.profile , builtin:management-zones
classic	Classic Config API v1	Auto-tag rules, request attributes
documents	Dashboards, notebooks	Document API objects
automations	Workflows	Automation API objects
buckets	Grail buckets	Bucket definitions
segments	Filter segments	Segment definitions
openpipeline	OpenPipeline configs	Pipeline processing rules

Note: Some classic API types (AWS/Azure/K8s credentials, Extensions v1) support deploy but **not** download. See the [Monaco API coverage](#) for details.

5. Explore Downloaded Configuration

After downloading, examine the directory structure and understand the file format.

Directory Structure

```
# View the downloaded directory structure
cd dynatrace-config
find downloaded -type f | head -30
```

Expected output looks like:

```
downloaded/
  my-project/
    builtin:alerting.profile/
      config.yaml
      ap-production-alerts.json
    builtin:management-zones/
      config.yaml
      mz-production.json
    builtin:tags.auto-tagging/
      config.yaml
      tag-environment.json
    ...
```

Example config.yaml

Each config type directory contains a `config.yaml` that references JSON template files:


```
configs:
  - id: ap-production-alerts
    config:
      name: "Production Alerts"
      template: ap-production-alerts.json
    type:
      settings:
        schema: builtin:alerting.profile
        scope: environment
```

Key Fields

Field	Description
<code>id</code>	Unique identifier for this config within the project
<code>config.name</code>	Display name in the Dynatrace UI
<code>config.template</code>	Path to the JSON payload file
<code>type.settings.schema</code>	The Settings 2.0 schema ID
<code>type.settings.scope</code>	Deployment scope (<code>environment</code> , <code>host</code> , <code>service</code> , etc.)

Examine a JSON Template

```
# View an example JSON template (replace path with an actual downloaded file)
cd dynatrace-config
cat downloaded/my-project/builtin:alerting.profile/ap-production-alerts.json
| python3 -m json.tool | head -30
```

Tip: The JSON templates contain the exact payload that Monaco sends to the Dynatrace API. You can modify these files to change configuration values.

6. Create a New Configuration

Create a new alerting profile from scratch to understand how Monaco configurations are authored.

Step 1: Create the Directory

```
# Create directory for alerting profile configs
mkdir -p dynatrace-config/my-project/alerting-profiles
```

Step 2: Create config.yaml

Create `dynatrace-config/my-project/alerting-profiles/config.yaml` :

```
configs:
  - id: production-critical-alerts
    config:
      name: "Production Critical Alerts"
      template: production-critical-alerts.json
    type:
      settings:
        schema: builtin:alerting.profile
        scope: environment
```

Step 3: Create the JSON Template

Create `dynatrace-config/my-project/alerting-profiles/production-critical-alerts.json` :

```

{
  "name": "{{ .name }}",
  "severityRules": [
    {
      "severityLevel": "AVAILABILITY",
      "delayInMinutes": 0,
      "tagFilterIncludeMode": "INCLUDE_ANY"
    },
    {
      "severityLevel": "ERRORS",
      "delayInMinutes": 5,
      "tagFilterIncludeMode": "INCLUDE_ANY"
    },
    {
      "severityLevel": "PERFORMANCE",
      "delayInMinutes": 15,
      "tagFilterIncludeMode": "INCLUDE_ANY"
    },
    {
      "severityLevel": "RESOURCE_CONTENTION",
      "delayInMinutes": 30,
      "tagFilterIncludeMode": "INCLUDE_ANY"
    }
  ]
}

```

How References Work

Configurations can reference each other using the `{{ .project-name.config-id.id }}` syntax:

```

configs:
- id: notification-with-profile
  config:
    name: "Slack Notification"
    template: slack-notification.json
    parameters:
      alerting_profile_id: "{{ .alerting-profiles.production-critical-alerts.id }}"
    type:
      settings:
        schema: builtin:problem.notifications
        scope: environment

```

Monaco resolves these references during deployment, ensuring the correct runtime IDs are used.

7. Validate Configuration

Always validate before deploying. The `--dry-run` flag checks everything without making changes.

Run Dry-Run Validation

```
# Validate all configurations without deploying
cd dynatrace-config
monaco deploy --dry-run manifest.yaml
```

What Dry-Run Checks

Check	Description
YAML syntax	Valid YAML in <code>manifest.yaml</code> and <code>config.yaml</code> files
JSON syntax	Valid JSON in template files
Schema validation	Template matches the Settings 2.0 schema
Reference resolution	Cross-config references (<code>{{ .ref }}</code>) resolve correctly
Environment connectivity	Can reach the target environment
Token permissions	Token has required scopes

Common Validation Errors

Error	Cause	Fix
<code>template not found</code>	Wrong path in <code>config.template</code>	Use path relative to <code>config.yaml</code>
<code>unknown schema</code>	Typo in schema ID	Verify schema at Settings > Schema in Dynatrace

Error	Cause	Fix
unresolved reference	Referenced config ID does not exist	Check spelling and project path
invalid JSON	Syntax error in template	Run <code>python3 -m json.tool <file></code> to find the error
insufficient permissions	Token missing scopes	Add required scopes to the API token

8. Deploy Configuration

After validation passes, deploy to your target environment.

Deploy to All Environments

```
# Deploy to all environments defined in manifest.yaml
cd dynatrace-config
monaco deploy manifest.yaml
```

Deploy to a Specific Environment

```
# Deploy only to the dev environment
cd dynatrace-config
monaco deploy manifest.yaml --environment dev
```

Expected Output

```
2026-04-04T10:00:00Z INFO Loading manifest "manifest.yaml"
2026-04-04T10:00:00Z INFO Projects to deploy: my-project
2026-04-04T10:00:01Z INFO Deploying config production-critical-alerts
(builtin:alerting.profile) to dev...
2026-04-04T10:00:02Z INFO Deployed 1/1 configs successfully
```

Verify in Dynatrace

After deployment:

1. Navigate to **Settings > Alerting > Alerting profiles** in the Dynatrace UI
2. Confirm the "Production Critical Alerts" profile appears
3. Verify the severity rules match your JSON template

Deploy a Specific Project

```
# Deploy only a specific project (useful when manifest has multiple projects)
cd dynatrace-config
monaco deploy manifest.yaml --project my-project
```

Tip: Monaco uses the config `id` as an idempotency key. Re-deploying the same config updates it in place rather than creating duplicates.

9. Delete Configuration

Monaco can remove configurations that are no longer needed.

Create a delete.yaml File

Create `dynatrace-config/delete.yaml` :

```
delete:
  - project: my-project
    type: builtin:alerting.profile
    id: production-critical-alerts
```

Execute Delete

```
# Delete configurations listed in delete.yaml
cd dynatrace-config
monaco delete manifest.yaml --file delete.yaml --environment dev
```

Delete Behavior

Behavior	Description
Scoped to Monaco	Only deletes configs that Monaco deployed (matched by <code>id</code>)
Non-destructive	Configs created manually in the UI are never touched
Reversible	Re-deploy the config to restore it
Environment-specific	Use <code>--environment</code> to target a single environment

Warning: The `delete` command is irreversible for the target environment. Always use `--dry-run` first if you want to preview what would be removed.

10. Multi-Environment Promotion

A core Monaco strength is deploying the same configuration across dev, staging, and production with environment-specific overrides.

Multi-Environment Manifest

```
manifestVersion: "1.0"

projects:
  - name: my-project
    path: my-project

environmentGroups:
  - name: default
    environments:
      - name: dev
        url:
          type: environment
          value: DT_DEV_URL
        auth:
          token:
            name: DT_DEV_TOKEN
      - name: staging
        url:
          type: environment
          value: DT_STAGING_URL
        auth:
          token:
            name: DT_STAGING_TOKEN
      - name: prod
        url:
          type: environment
          value: DT_PROD_URL
        auth:
          token:
            name: DT_PROD_TOKEN
```

Environment-Specific Overrides

Use `overrides` in `config.yaml` to customize values per environment:


```
configs:
  - id: alerting-profile
    config:
      name: "Alerting Profile"
      template: alerting.json
      parameters:
        delay_minutes: 15
      overrides:
        - environments:
            - staging
          override:
            parameters:
              delay_minutes: 10
        - environments:
            - prod
          override:
            parameters:
              delay_minutes: 0
    type:
      settings:
        schema: builtin:alerting.profile
        scope: environment
```

Promotion Workflow

```
# Step 1: Deploy to dev and validate
cd dynatrace-config
monaco deploy manifest.yaml --environment dev

# Step 2: Deploy to staging after dev validation
monaco deploy manifest.yaml --environment staging

# Step 3: Deploy to production after staging validation
monaco deploy manifest.yaml --environment prod
```

Skip Configs in Certain Environments

Use the `skip` field to exclude configurations from specific environments:

```
configs:
  - id: debug-dashboard
    config:
      name: "Debug Dashboard"
      template: debug-dashboard.json
      skip:
        environments:
          - prod
    type:
      document:
        kind: dashboard
```

This deploys the debug dashboard to dev and staging but skips production entirely.

11. GitHub Actions CI/CD Pipeline

Automate Monaco deployments with a GitHub Actions workflow that validates on pull requests and deploys on merge to main.

Workflow File

Create `.github/workflows/monaco-deploy.yml` :

```

name: Deploy Dynatrace Config

on:
  push:
    branches: [main]
    paths: ['dynatrace-config/**']
  pull_request:
    branches: [main]
    paths: ['dynatrace-config/**']

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install Monaco
        run: |
          curl -L https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/latest/download/monaco-linux-amd64 -o monaco
          chmod +x monaco
          sudo mv monaco /usr/local/bin/

      - name: Validate (dry-run)
        run: monaco deploy --dry-run dynatrace-config/manifest.yaml
        env:
          DT_ENV_URL: ${ secrets.DT_ENV_URL }
          DT_API_TOKEN: ${ secrets.DT_API_TOKEN }

  deploy:
    needs: validate
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install Monaco
        run: |
          curl -L https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/latest/download/monaco-linux-amd64 -o monaco
          chmod +x monaco
          sudo mv monaco /usr/local/bin/

      - name: Deploy to Dynatrace
        run: monaco deploy dynatrace-config/manifest.yaml
        env:

```

```
DT_ENV_URL: ${ secrets.DT_ENV_URL }
DT_API_TOKEN: ${ secrets.DT_API_TOKEN }
```

Workflow Explanation

Step	Trigger	What Happens
validate	Every PR and push to <code>main</code>	Installs Monaco, runs <code>--dry-run</code> to catch errors
deploy	Only on push to <code>main</code>	Installs Monaco, deploys all configurations

Configure GitHub Secrets

Add these secrets to your GitHub repository (**Settings > Secrets and variables > Actions**):

Secret Name	Value
<code>DT_ENV_URL</code>	Your Dynatrace tenant URL (e.g., <code>https://abc12345.live.dynatrace.com</code>)
<code>DT_API_TOKEN</code>	API token with Configuration as Code scopes

Multi-Environment CI/CD

For promoting across environments, extend the workflow with separate jobs:

```

deploy-staging:
  needs: deploy
  runs-on: ubuntu-latest
  environment: staging
  steps:
    - uses: actions/checkout@v4
    - name: Install Monaco
      run: |
        curl -L https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/latest/download/monaco-linux-amd64 -o monaco
        chmod +x monaco
        sudo mv monaco /usr/local/bin/
    - name: Deploy to Staging
      run: monaco deploy dynatrace-config/manifest.yaml --environment staging
  env:
    DT_STAGING_URL: ${ secrets.DT_STAGING_URL }
    DT_STAGING_TOKEN: ${ secrets.DT_STAGING_TOKEN }

```

Use GitHub [Environments](#) with required reviewers to gate production deployments.

12. Summary & Checklist

Deployment Checklist

- [] Monaco CLI installed and `monaco version` returns output
- [] API token created with "Configuration as Code" template scopes
- [] `manifest.yaml` created with correct environment variable references
- [] Downloaded existing configuration with `monaco download`
- [] Explored directory structure and understood `config.yaml` format
- [] Created a new configuration (alerting profile) from scratch
- [] Validated with `monaco deploy --dry-run` (no errors)
- [] Deployed to target environment with `monaco deploy`
- [] Verified deployed config in Dynatrace UI
- [] Tested delete workflow with `delete.yaml`
- [] Configured multi-environment promotion with overrides
- [] Created GitHub Actions workflow for CI/CD

Key Takeaways

Concept	Summary
Manifest	Entry point defining projects and environments
Config Types	Settings, classic, documents, automations, buckets, segments, openpipeline
Download	Export existing tenant config to versioned files
Dry-Run	Always validate before deploying
Deploy	Idempotent -- re-running updates in place
Delete	Only removes Monaco-managed configs
Overrides	Environment-specific parameter values
CI/CD	Validate on PR, deploy on merge to main

References

- [Monaco Documentation](#)
- [Monaco GitHub Repository](#)
- [Monaco API Coverage](#)
- [Settings Schema Reference](#)

Return to AUTOM-03: Monaco Configuration-as-Code for the lecture content, or continue to AUTOM-04: Terraform Provider.

Community Resources

Repository	Description
dynatrace-configuration-as-code	Official Monaco CLI (v2.28.5)
dynatrace-configuration-as-code-samples	9 starter templates in <code>basic-templates-monaco</code> , plus pipeline observability samples

Repository	Description
easytrade	Real-world Monaco project structure with manifest.yaml
Dynatrace-Config-Manager	GUI tool for tenant-to-tenant config migration
monaco-self-paced-exercises	6 structured exercises (install, auto-tag, download, variables, delete/restore, linking)
monaco-demo	Working GitHub Actions workflow for Monaco deploy

Community Resources & Examples

After completing the hands-on steps above, these repositories provide deeper examples and exercises to reinforce what you've practiced:

Official Repositories

Repository	Description
dynatrace-configuration-as-code	Official Monaco CLI (v2.28.5) — Go binary, Apache-2.0
dynatrace-configuration-as-code-samples	Official samples repo with 9 Monaco starter templates in <code>basic-templates-monaco</code>
easytrade	Demo microservices app with a working <code>monaco/</code> directory (manifest.yaml, detection rules, workflows)
Dynatrace-Config-Manager	GUI tool for tenant-to-tenant config migration; complements Monaco for brownfield scenarios

Training & Exercises (recommended next)

Repository	Description
monaco-self-paced-exercises	6 structured exercises: install, auto-tag, download, variables, delete/restore, linking configs
monaco-demo	Working GitHub Actions workflow for Monaco deploy with "crawl-walk-run" adoption methodology

Pipeline Observability Samples

Monaco configurations for ingesting CI/CD pipeline events via OpenPipeline:

Directory	CI/CD Platform
github_pipeline_observability	GitHub Actions
gitlab_pipeline_observability	GitLab CI
azure_devops_observability	Azure DevOps
argocd_observability	ArgoCD

Note: Monaco v1 (`dynatrace-oss/dynatrace-monitoring-as-code`) is deprecated.
Use `monaco convert` to migrate v1 projects to v2.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.